

ZERO KNOWLEDGE PROOFS

www.matematickaakademija.com

Content

- Circom installation
- Generating proofs with snarkjs
- PLONK vs Groth16

Circom installation: macOS / Linux

1. **Rust toolchain** installs rustup + cargo, then open a new terminal

```
curl --proto '=https' --tlsv1.2 https://sh.rustup.rs -sSf | sh
```

2. **Clone circom**

```
git clone https://github.com/iden3/circom.git
```

3. **Build & install** takes a few minutes

```
cd circom && cargo build --release && cargo install --path circom
```

4. **snarkjs** JavaScript library, needs Node.js

```
npm install -g snarkjs
```

5. **Check**

```
circom --help
```

Needs Node.js, git and a C compiler (macOS: xcode-select --install, Linux: build-essential). Full guide

Circom installation: Windows

1. **Rust toolchain** download and run rustup-init.exe, accept Visual Studio C++ Build Tools

```
https://rustup.rs
```

2. **Install circom** downloads, builds and installs, no clone needed

```
cargo install --locked --git https://github.com/iden3/circom circom
```

3. **snarkjs** JavaScript library, needs Node.js

```
npm install -g snarkjs
```

4. **Check** in a new PowerShell window

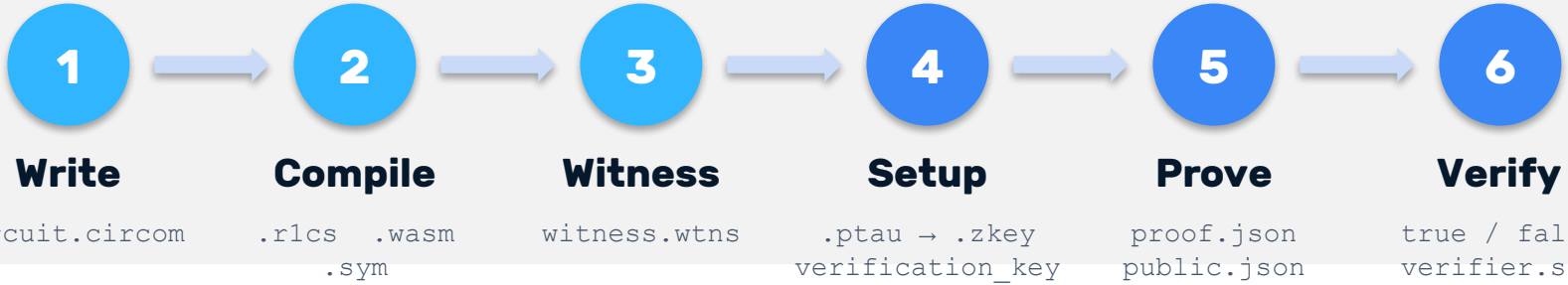
```
circom --help
```

5. **Run .ps1 scripts** Windows blocks them by default

```
powershell -ExecutionPolicy Bypass -File .\script-groth16.ps1
```

Run the commands in PowerShell. Full guide

Process of proving circuits



circom compiler + generated WASM

Circuit becomes a constraint system (R1CS). The WASM program computes the values of all signals from inputs.json.

snarkjs (Groth16 or PLONK)

Setup produces proving/verification keys. Prover uses .zkey + witness; verifier needs only the vkey and public signals.

Example: guess the number

```
template guessTheNumber() {
    signal input number;      // public
    signal input solution;    // private
    signal input commit;      // public
    signal input salt;        // private
    signal output out;

    component commit_calc = Poseidon(2);
    commit_calc.inputs[0] <== solution;
    commit_calc.inputs[1] <== salt;
    commit === commit_calc.out;

    component eq = IsEqual();
    eq.in[0] <== number;
    eq.in[1] <== solution;
    out <== eq.out;
}
```

Idea

Secret number hidden behind a Poseidon commitment; prove whether a guess is right without revealing it.

Constraint

`commit === Poseidon(solution, salt)`

Output

`out = 1  iff  number == solution`

Operators

- `<==` assign + constrain
- `===` constrain only
- `<--` assign only (unsafe alone)

Also in circuit.circom: `include` poseidon + comparators from circomlib, `component main {public [number, commit]}`

Compile & compute the witness

1. **Dependencies** circomlib, snarkjs, poseidon-lite

```
npm install
```

2. **Inputs** computes the commitment, writes inputs.json

```
node generate-inputs.js
```

3. **Compile** .r1cs constraints • .wasm witness program • .sym signal names

```
mkdir -p artifacts && circom circuit.circom --r1cs --wasm --sym --output artifacts
```

4. **Witness** run in artifacts/ → witness.wtns: values of all signals, stays private

```
node circuit_js/generate_witness.js circuit_js/circuit.wasm ../inputs.json witness.wtns
```

Trusted setup: powers of tau

Phase 1 • universal

- Independent of the circuit, depends only on max size: 2^12 = 4096 constraints
- Multi-party ceremony: secure if at least one contributor is honest
- Can reuse a public ceremony (perpetual powers of tau)

```
snarkjs powersoftau
new → contribute → beacon
→ prepare phase2
```

Phase 2 • circuit-specific

- Depends on circuit.r1cs, redo it every time the circuit changes
- Needed only for Groth16
- PLONK skips it: plonk setup uses the phase 1 file directly

```
snarkjs groth16 setup
→ snarkjs zkey contribute
```

snarkjs docs: github.com/iden3/snarkjs • perpetual powers of tau

Proving with Groth16

1. **Setup** circuit-specific, uses a public phase 1 file

```
snarkjs groth16 setup circuit.r1cs powersOfTau28_hez_final_12.ptau circuit_0000.zkey
```

2. **Contribute** phase 2 ceremony, asks for random text

```
snarkjs zkey contribute circuit_0000.zkey circuit_final.zkey --name="mihailo" -v
```

3. **Verification key**

```
snarkjs zkey export verificationkey circuit_final.zkey verification_key.json
```

4. **Prove** proof.json: only 3 curve points • public.json: [out, number, commit]

```
snarkjs groth16 prove circuit_final.zkey witness.wtns proof.json public.json
```

5. **Verify** prints OK! • on-chain: zkey export solidityverifier → verifier.sol

```
snarkjs groth16 verify verification_key.json public.json proof.json
```

Proving with PLONK

1. **Phase 1** our own ceremony: then contribute ×3, beacon, prepare phase2

```
snarkjs powersoftau new bn128 12 pot12_0000.ptau -v
```

2. **Setup** universal: no circuit-specific contribution

```
snarkjs plonk setup circuit.r1cs pot12_final.ptau circuit_final.zkey
```

3. **Verification key**

```
snarkjs zkey export verificationkey circuit_final.zkey verification_key.json
```

4. **Prove** proof.json: 9 curve points + 6 field elements

```
snarkjs plonk prove circuit_final.zkey witness.wtns proof.json public.json
```

5. **Verify** prints OK! • on-chain: zkey export solidityverifier → verifier.sol

```
snarkjs plonk verify verification_key.json public.json proof.json
```

Proving in JavaScript

```
const { groth16 } = require("snarkjs");  // or plonk
const inputs = JSON.parse(readFileSync("inputs.json"));

const { proof, publicSignals } = await groth16.fullProve(
    inputs,
    "./artifacts-groth16/circuit_js/circuit.wasm",
    "./artifacts-groth16/circuit_final.zkey");

const vKey = JSON.parse(
    readFileSync("./artifacts-groth16/verification_key.json"));
const ok = await groth16.verify(vKey, publicSignals, proof);
console.log(ok ? "Verification OK" : "Invalid proof");
```

fullProve
computes the witness and the proof in one step

Same API
plonk.fullProve / plonk.verify with the PLONK artifacts

Client-side
runs in the browser: private inputs never leave the user

PLONK vs Groth16

	PLONK	Groth16
Trusted setup	universal (phase 1 only)	circuit specific (phase 1 + 2)
Proof size	bigger: 9 points + 6 field el.	smaller: 3 points
Proving time	longer	shorter
Verification	longer	shorter
New circuit	reuse the same .ptau	new phase 2 ceremony

Groth16: many proofs for the same circuit, performance critical. **PLONK**: many different circuits (arbitrary business logic) with reasonably fast performance.



Thank you!